CSI 3004 – Text Mining

CSI 3004 – Text Mining

Module:1	Information Extraction and Text Summarization	7 hours
Information Extraction - Named Entity Recognition - Relation Extraction - Unsupervised Information Extraction; Text Summarization - Topic Representation Approaches - Indicator Representations and Machine Learning.		

What is Text Mining ?

Text mining is the science of transforming raw text into meaningful insights.

Why Text Mining ?

- Almost **80 percent of all data in the world is unstructured**, and most of that is text: emails, tweets, reviews, medical reports, legal documents, research articles, and more.

- Understanding this unstructured information is now critical for industries ranging from health care and finance to retail and cyber security.

What Text Mining Actually Does?

Machines cannot understand language directly; We must convert text into numerical representations & extract patterns.

Key capabilities to explore (But not limited to):

- Identifying important words and topics from documents.
- Classifying text into categories like spam or non spam.
- Extracting opinions and emotions from social media posts.
- Discovering relationships / patterns hidden in text corpora.
- Summarizing long documents into a few sentences.

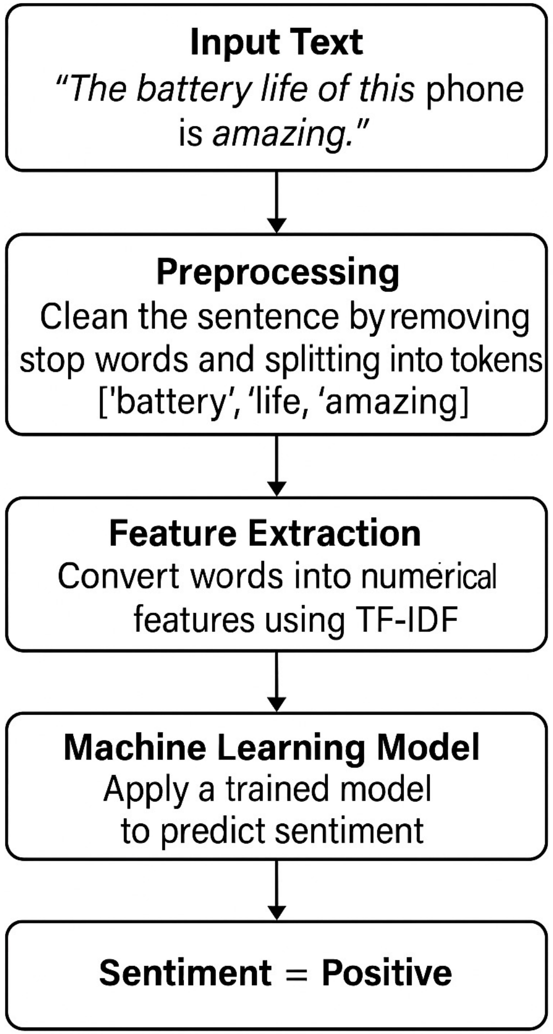
Example

“The battery life of this phone is amazing.”

Example – (How Text Mining Predicts Sentiment)

“The battery life of this phone is amazing.”

Text mining transforms raw text into numerical features, uses learned patterns from past data, and automatically identifies that words like “amazing” express a positive sentiment.



Introduction - What is Information Extraction (IE)?

- IE is the process of **automatically identifying key facts** from raw text.
- Converts text into **entities**, **relations**, and **events**.

Example:

"Dr. Meera visited Apollo Hospital on Monday."

Entity: Dr. Meera ; Entity: Apollo Hospital

Relation: visited. ; Event: medical consultation

- **Entity:** The main actors or objects in the text.
- **Relation:** The connection between those entities.
- **Event:** The action or occurrence that involves the entities.

Introduction - What is Information Extraction (IE)?

Example :
"Google acquired DeepMind in 2014 to advance artificial intelligence research."

Entities: Google, DeepMind, 2014, artificial intelligence research
Relations: acquired, aimed at
Event: Acquisition for AI research expansion

Introduction - Why we need Information Extraction?

Why ?

- The web generates billions of text documents every day; humans cannot manually read or organize them.
- IE turns **unstructured text** into **structured, analyzable knowledge**.
- IE powers real-world systems: chatbots, search engines, fraud detectors, medical decision systems.

Introduction - Why Information Extraction Important?

Why IE Matters ?

- ❖ Helps machines “understand” language.
- ❖ Enables deeper text analytics, knowledge graphs, intelligent search, and automation.

Introduction - Difference Between IR, NLP and IE

Task	Purpose	Example
IR (Information Retrieval)	Finding relevant documents	Google search for "COVID symptoms"
NLP (Natural Language Processing)	Understanding language structure and meaning	POS tagging, parsing, sentiment
IE (Information Extraction)	Pulling out *facts* and *relationships*	Extracting disease names from articles

How They Work Together ?

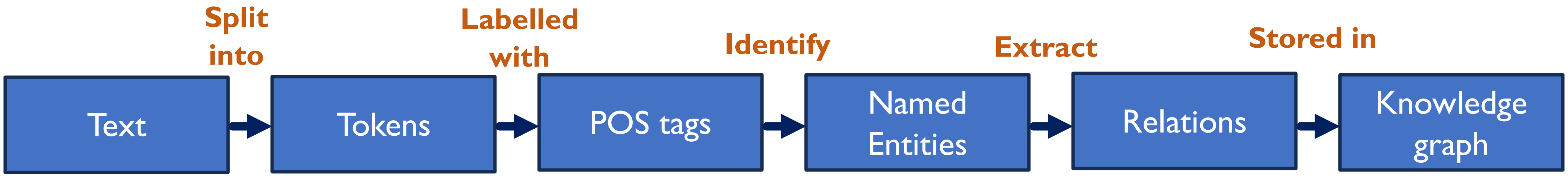
- ❖ **NLP** provides linguistic tools (tokenization, POS tags).
- ❖ **IR** retrieves documents from massive corpora.
- ❖ **IE** extracts meaningful knowledge from those documents.
- ❖ Together they form the backbone of modern intelligent systems.

Introduction - IE Pipeline Overview

Preprocessing → **Annotation** → **Extraction** → **Evaluation**

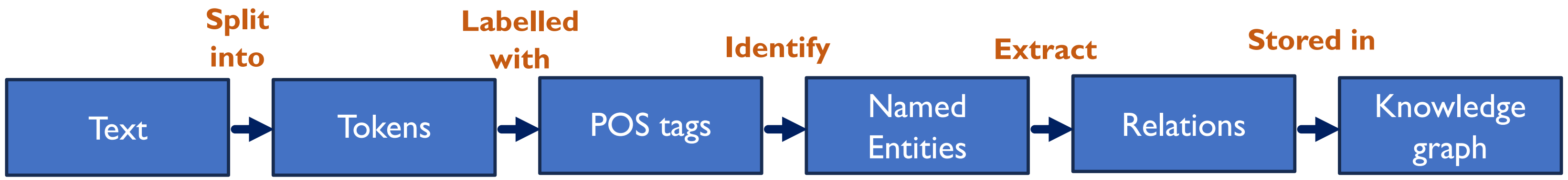
Preprocessing	Annotation	Extraction	Evaluation
Clean the text (tokenize, normalize, remove noise).	Label words with linguistic information (POS, Named Entity Recognition (NER tags)).	Identify entities, relations, events, facts.	Measure accuracy (precision, recall, F1 score).

Introduction - Visual Pipeline of IE



A sentence is gradually converted into structured knowledge by identifying words, tagging them, finding entities, discovering relations, and building a knowledge graph.

Introduction - Visual Pipeline of IE (Example)



“Virat Kohli scored 82 runs at the MCG during the 2022 T20 World Cup.”

Text

“Virat Kohli scored 82 runs at the MCG during the 2022 T20 World Cup.”

Tokens

["Virat", "Kohli", "scored", "82", "runs", "at", "the", "MCG", "during", "the", "2022", "T20", "World", "Cup"]

Introduction - Visual Pipeline of IE (Example)

Text $\xrightarrow{\text{Split into}}$ Tokens $\xrightarrow{\text{Labelled with}}$ POS tags $\xrightarrow{\text{Identify}}$ Named Entities $\xrightarrow{\text{Extract}}$ Relations $\xrightarrow{\text{Stored in}}$ Knowledge graph

“Virat Kohli scored 82 runs at the MCG during the 2022 T20 World Cup.”

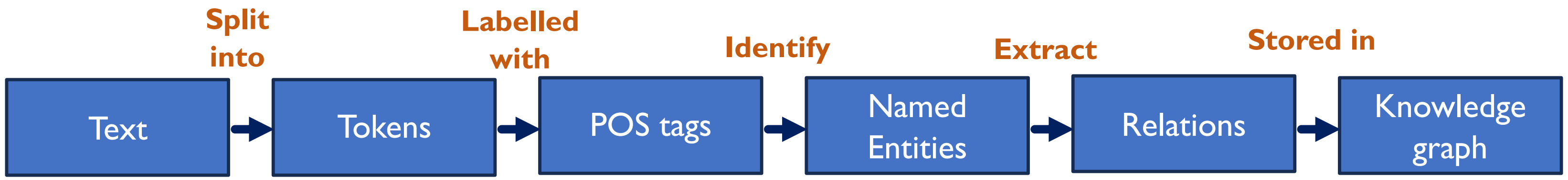
POS Tags

- Virat → Noun
- Kohli → Noun
- scored → Verb
- 82 → Number
- MCG → Proper noun
- 2022 → Number
- World Cup → Proper noun phrase

Named Entities

- **Virat Kohli** → Person
- **MCG** → Location
- **2022 T20 World Cup** → Event
- **82 runs** → Performance metric

Introduction - Visual Pipeline of IE (Example)



"Virat Kohli scored 82 runs at the MCG during the 2022 T20 World Cup."

Relations

scored → relation between Virat Kohli and 82 runs
(Virat Kohli → scored → 82 runs)
at → relation between action and location
(scored → at → MCG)
during → relation between action and event
(scored → during → 2022 T20 World Cup)

Knowledge Graph Output

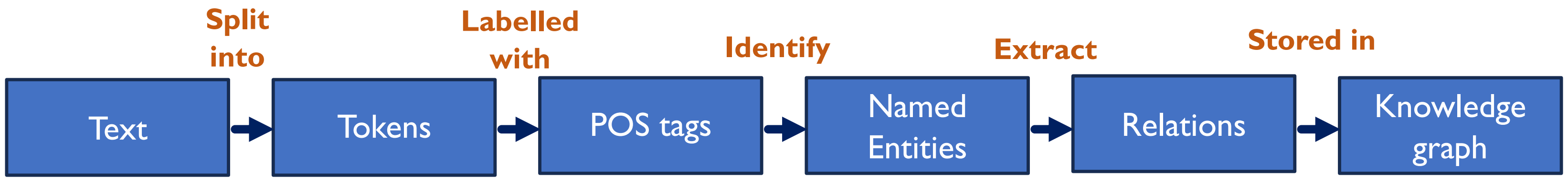
A structured graph would contain edges such as:
Virat Kohli -scored→ 82 runs
Virat Kohli -played at→ MCG
Performance -during→ 2022 T20 World Cup

Introduction - Visual Pipeline of IE (Example)

Text $\xrightarrow{\text{Split into}}$ Tokens $\xrightarrow{\text{Labelled with}}$ POS tags $\xrightarrow{\text{Identify}}$ Named Entities $\xrightarrow{\text{Extract}}$ Relations $\xrightarrow{\text{Stored in}}$ Knowledge graph

"Dr. Arjun prescribed Metformin to a diabetic patient at Apollo Clinic last Friday."

Introduction - Visual Pipeline of IE (Example)



"Sara tweeted that the new iPhone 16 camera is amazing and tagged Apple in her post.

Introduction - Applications of Information Extraction

Healthcare

Social Media Analytics

Cyber Security

Business Intelligence in Finance

Introduction : Future Scope and Career Relevance

- ❖ IE is at the heart of search engines, voice assistants, recommender systems, and generative AI.
- ❖ Every industry needs systems that convert text into actionable insights.

Skills in IE open pathways to careers in:

- ❖ Data science
- ❖ AI engineering
- ❖ Information security
- ❖ Health informatics
- ❖ Knowledge graph engineering

Named Entity Recognition (NER)

Named Entity Recognition (NER)

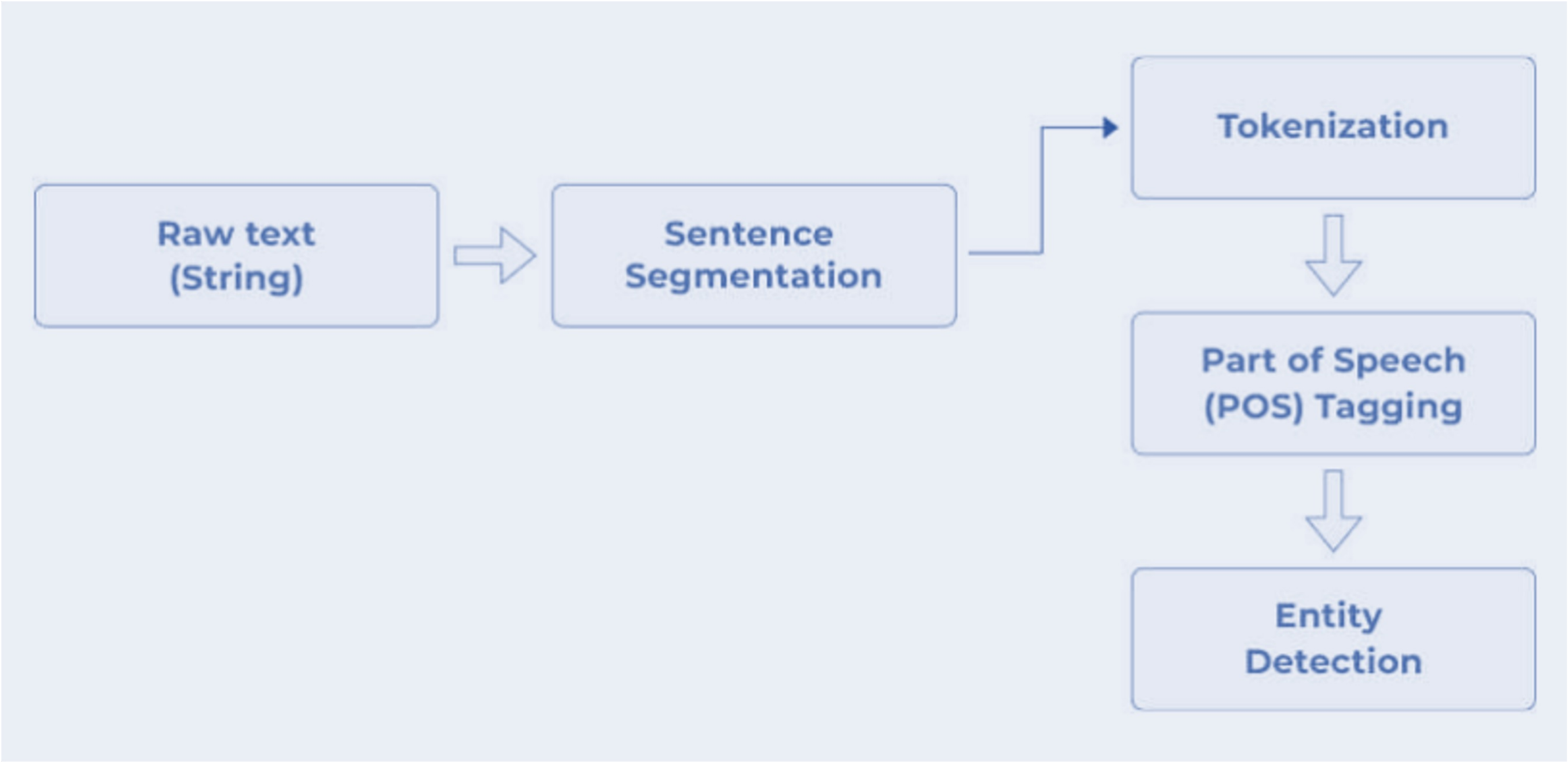
NER is the task of **automatically identifying important real-world entities** in text.

- Extracts "*who, where, what, when*" from unstructured text.
- Key step in transforming text into structured knowledge used by search engines, chatbots, RS, etc...

Example:

Text: "Elon Musk visited Bengaluru in January."
NER Output: Elon Musk (Person), Bengaluru (Location), January (Date)

NER Process Steps



Why NER is Important ?

- Reduces manual annotation effort by automatically identifying key elements.
- Helps build **knowledge graphs**, **question answering systems**, and **summarization engines**.
- Enables intelligent applications in **healthcare**, **finance**, **customer support**, and **cybersecurity**.

NER - Common Entity Types

Person – “Virat Kohli”, “Dr. Meera”

Organization – “Google”, “WHO”, “Apollo Hospital”

Location – “Delhi”, “Japan”, “MCG Stadium”

Date / Time – “2022”, “last Friday”

Product – “iPhone 16”

Event – “Olympics”, “T20 World Cup”

Medical Entities – diseases, drugs, symptoms (important in healthcare IE)

NER - Common Entity Types

"Tesla's new autonomous vehicle just completed a 1000-mile journey across the US with zero driver intervention. #AutonomousVehicles #Tesla"

- **Tesla → Organization**
- **autonomous vehicle → Product**
- **1000-mile journey → Event**
- **US → Location**

How NER Works – Approaches ?

A. Rule Based NER

- Uses handwritten patterns, dictionaries, regular expressions.
- Good for well structured text (ex: medical prescriptions).
- Limitation: hard to scale and maintain.

B. Statistical NER

- Uses probabilistic models trained on annotated datasets.
- Popular early method: **Conditional Random Fields (CRF)**.
- Learns patterns - capital letters, context words, part of speech.

C. Deep Learning NER

- Current state-of-the-art.
- Learns contextual meaning from millions of sentences.
- Handles slang, typos, abbreviations better.

How CRF-Based NER Works ?

CRF-Based NER

- CRF models predict entity labels by considering relationships between neighboring words.
- Effective for sequence labeling tasks.
- Still widely used for domain-specific NER (legal, biomedical).

Example:

Tokens: [Dr., Meera, visited, Apollo, Hospital]
Labels: [Title, Person, O, Organization, Organization]

"O". - represents outside (Not an entity)

How Neural NER Works ?

Neural NER – BiLSTM-CRF

- Combines:
- **BiLSTM** – learns long-term context from both left and right directions.
- **CRF** – ensures coherent label sequences.

- Significantly improves accuracy in noisy text and social media content.

BiLSTM reads sentences like humans: from left to right and right to left to understand meaning.

How Transformer-Based NER Works?

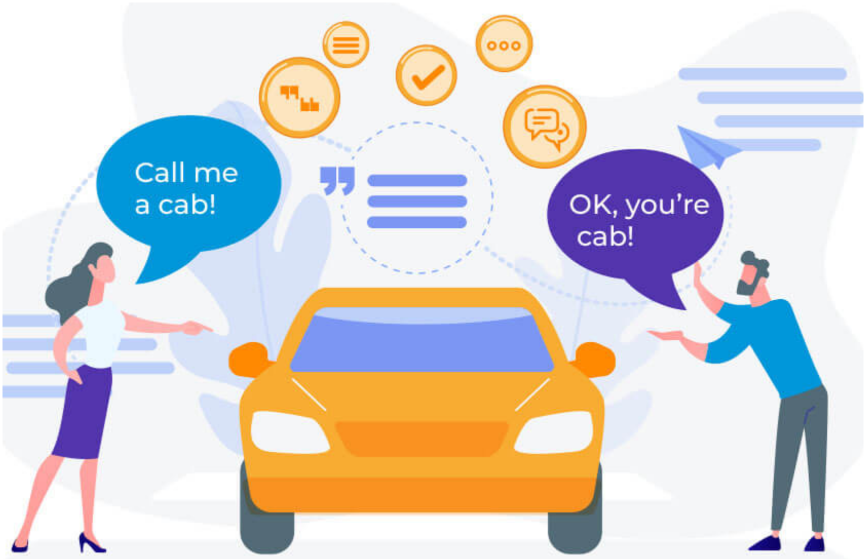
Transformer-Based NER (Modern Approach)
Models like **BERT**, **DistilBERT** revolutionized NER.
Key strengths:

- Understand context deeply (ex: "Apple" as fruit vs company).
- Handle domain adaptation better.
- Work extremely well for large scale IE and knowledge graph construction.

Sentence: "Apple announced new products in California."
BERT knows "Apple" = Company, not fruit.

NER Challenges

1. Ambiguity. (Lexical / Syntactic)
2. Domain Adaptation
3. Multilingual NER
4. Noisy Text



Relation Extraction (RE)

Relation Extraction (RE)

- Relation Extraction (RE) is the process of identifying and classifying relationships between entities in text.
- For example, recognizing that "Virat Kohli" **plays for** "Royal Challengers Bangalore".

RE allows systems to construct structured knowledge from unstructured text, which is crucial knowledge graph creation & information retrieval.

“Elon Musk founded SpaceX in 2002.”
Entities: “Elon Musk” (Person), “SpaceX” (Organization), “2002” (Date)

Relation Extraction (RE) - Types of Relations

Types of Relations

Binary Relations: The simplest form of relations, involving two entities.

Example: "Albert Einstein" **was born in** "Ulm."

N-ary Relations: Relations involving more than two entities.

Example: "Apple Inc." **released** "iPhone 12" **on** "October 13, 2020."

Binary relations are more commonly extracted than n-ary, though n-ary relations are crucial for capturing complex interactions.

Relation Extraction (RE) - Types of Relations

- Classify a sentence: "Bill Gates co-founded Microsoft with Paul Allen in 1975"

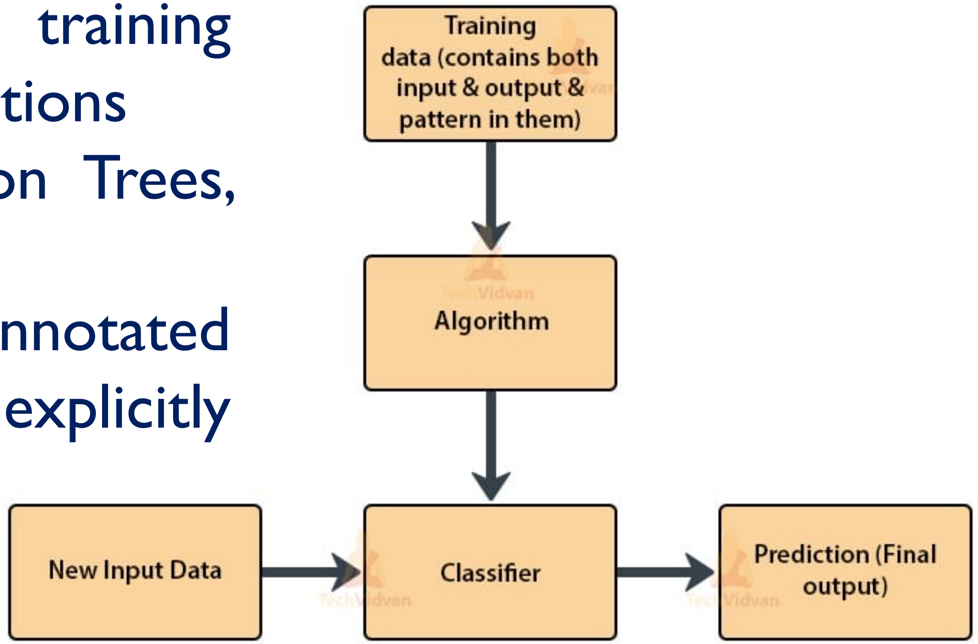
  Entities: Bill Gates (Person), Microsoft (Organization), Paul Allen (Person), 1975 (Date)

  Relation: Co-founded

- Whether this is binary or n-ary ?

Relation Extraction (RE) – Supervised ML

- **Supervised Learning**: Involves training models on labeled data to classify relations
- Common algorithms include Decision Trees, SVM, and Neural Networks.
- **Training Process**: Requires annotated corpora where relationships are explicitly labeled.



- Supervised models can be highly accurate but require large amounts of labeled data, which can be a limitation. **Example - T**ry to build a simple classifier.

Neural Approaches for Relation Extraction (RE)

- **Convolutional Neural Networks (CNN)**: Used for extracting local patterns in the text to identify relations - used in sentence classification tasks.
- **Recurrent Neural Networks (RNN)**: Effective for modeling sequences of words & capturing contextual dependencies between words for relation extraction.
- **Transformers**: Modern architectures like BERT, which are state-of-the-art for RE tasks - Handle long-range dependencies and context better than RNNs and CNNs.

Transformers has improved performance RE, with BERT-based models achieving superior performance over traditional methods.

Evaluation Metrics - Relation Extraction (RE)

		Predicted Class		
		Positive	**Negative**	
Actual Class	**Positive**	True Positive (TP)	False Negative (FN) **Type II Error**	**Sensitivity** $\frac{TP}{(TP + FN)}$
	Negative	False Positive (FP) **Type I Error**	True Negative (TN)	**Specificity** $\frac{TN}{(TN + FP)}$
		Precision $\frac{TP}{(TP + FP)}$	**Negative Predictive Value** $\frac{TN}{(TN + FN)}$	**Accuracy** $\frac{TP + TN}{(TP + TN + FP + FN)}$

Evaluation Metrics - Relation Extraction (RE)

Question: Consider a dataset with 10 sentences. After performing Relation Extraction (RE), the system identifies 6 relations (out of which 4 are correct, and 2 are incorrect). There are also 3 relations that the system missed.

True Positives (TP): The correct relations identified by the system (4).
False Positives (FP): Incorrect relations identified by the system (2).
False Negatives (FN): Correct relations that were not identified by the system (3).
True Negatives (TN): The number of cases where the system correctly identified no relation (0 relations, as all sentences had relations).

Evaluation Metrics - Relation Extraction (RE)

Using the provided information, calculate the following metrics for the Relation Extraction (RE) task:

- **Precision**
- **Recall**
- **F1 Score**
- **Accuracy**

- **True Positives (TP)** = 4
- **False Positives (FP)** = 2
- **False Negatives (FN)** = 3
- **True Negatives (TN)** = 0

Summary of the Results:

- **Precision** = 0.67
- **Recall** = 0.57
- **F1 Score** = 0.61
- **Accuracy** = 0.44

Relation Extraction (RE) – Use-cases

Knowledge Graphs:
RE helps build knowledge graphs by linking entities and their relations. These graphs are essential for tasks like search engines and recommendation systems.

Question Answering:
By extracting relations, RE helps answer complex questions by identifying the key entities and their relations in the context.

Explore how RE is foundational in advanced AI systems like virtual assistants (e.g., Siri, Alexa) and search engines (e.g., Google).

Unsupervised Information Extraction

Unsupervised Information Extraction - Introduction

What is Unsupervised Information Extraction (IE)?

- Unsupervised IE refers to extracting useful information (like entities, relations, or events) from text without labeled training data.

- It involves techniques where the system learns patterns and structures from raw data, without predefined labels, making it particularly useful when labeled data is scarce or expensive to acquire.

Unsupervised Information Extraction - Advantages

- **Limited Labeled Data**: In many cases, annotating large corpora is time-consuming and expensive.
- **Scalability**: Can scale better, especially in dynamic environments where new information is continuously generated.
- **Flexibility**: Can adapt to various domains without needing domain-specific labeled data.

Example:
Extracting relations like **"works_for"** between entities like **"Steve Jobs"** and **"Apple"** from raw news articles, without having labeled datasets for training.

Unsupervised Information Extraction - Introduction

Clustering-Based Information Extraction

- ❖ These methods group similar instances together based on patterns or features.
- ❖ Once the data is clustered, relationships within the same group can be inferred.
- ❖ Example: Cluster sentences containing similar named entities and extract relations based on co-occurrence patterns.

Unsupervised Information Extraction - Introduction

Types of Clustering for IE:

- **Entity Clustering**: Grouping similar entities together, such as companies, people, or locations.
- **Relation Clustering**: Identifying clusters of similar relations from text.

Key Concept:

Bag of Words Representation: Text is represented as a set of words without considering grammar or word order, simplifying the clustering process.

Example: Given a set of sentences, cluster them based on the similarity of the named entities mentioned, and identify the most common relations (e.g., **"Steve Jobs" founded "Apple"**).

Unsupervised Information Extraction - Steps

Step-by-Step Workflow for Unsupervised IE

Step 1: Data Collection

Goal: Collect unstructured data (e.g., product reviews).

Tool: **Scrapy** or **BeautifulSoup** for web scraping.

Step 2: Text Preprocessing

Goal: Clean and prepare data for extraction.

Steps:

- **Tokenization**: Break sentences into words (**NLTK**).
- **Stopword Removal**: Remove common words.
- **Stemming/Lemmatization**: Reduce words to their base form.

Unsupervised Information Extraction - steps

Step 3: Entity Recognition (Clustering) - Identify and group similar words (e.g., products, features).
Technique: Any **clustering** algo for grouping entities.
Example: Cluster "iPhone", "Samsung" as **Products** and "camera", "battery" as **Features**.

Step 4: Relation Extraction - Identify relationships between entities (e.g., "iPhone 13 - has_feature - camera").
Techniques:
Pattern-based Extraction: Identify patterns like "X such as Y"
OpenIE: Use frameworks **OpenIE** to extract triplets.
Example: Extract relation "Apple" **has_feature** "camera".

Unsupervised Information Extraction - steps

Step 5: Post-Processing & Structuring Data -
Organize extracted relations
Example: Store relations in a **Knowledge Graph** or DB.

Step 6: Evaluation - Assess performance using metrics like **Precision**, **Recall**, and **F1 Score**.
Example:
Precision = TP / (TP + FP)
Recall = TP / (TP + FN).

Unsupervised Information Extraction - (OpenIE) Frameworks

Open Information Extraction (OpenIE) Frameworks

- Unlike traditional IE systems, OpenIE does not assume a predefined set of relations.
- It extracts a wide variety of relations from text, making it flexible and suitable for many domains.

Features:

- Extracts **triplets**: **Subject - Relation - Object** (e.g., "Steve Jobs - founded - Apple").
- Does not require predefined relations or labeled data.
- OpenIE frameworks can operate on raw text and extract knowledge without domain-specific training.

Unsupervised Information Extraction - Challenges

- **Ambiguity**: Words or phrases may have multiple meanings, making it difficult to extract accurate relations.
- **Context**: The same relation might be expressed in many ways across different contexts.
- **Noise**: Unsupervised methods are often prone to errors due to noisy text or incorrect entity alignment.

Solution:Use clustering techniques, dependency parsing, or context-based models to reduce ambiguity.

Unsupervised Information Extraction - Use Case

Extracting Product-Related Information from E-commerce Reviews

Scenario:

- You work at an e-commerce company and want to extract useful information from customer reviews to understand the relations between products and features mentioned by customers.
- For example, you want to identify relations like **"product_name - has_feature - feature"** (e.g., **"iPhone 13 - has_feature - camera"**) from customer reviews.
- This is an unsupervised information extraction (IE) task, where you don't have labeled data to train the model. You want to use unsupervised algorithms to extract key information (like product names and their features) from thousands of customer reviews.

Text Summarization

Text Summarization - Introduction

- ❖ Text Summarization is the process of reducing a large body of text into a shorter version, maintaining key information and the essence of the original content.
- ❖ The goal is to create a concise summary while preserving critical facts and meaning.
- ❖ **Example**: This is especially useful in applications like news aggregation, research paper summarization, and customer review analysis

Types of Text Summarization

1. Extractive Summarization - Extracts key sentences, phrases, or passages directly from the source text to create a summary. No modification of the original sentences.

- **Example**: Selecting a sentence like "The iPhone 13's camera is the best ever" directly from the article as part of the summary.

2. Abstractive Summarization - Generates new sentences by understanding the meaning of the text and rephrasing it in a shorter form.

- **Example**: "The iPhone 13's camera has been improved significantly" (paraphrased).

Indicative vs Informative Summarization

- **Indicative Summarization**: Summarizes the theme of the document. It indicates what the text is about but doesn't provide much detailed information.
- **Example**: "This article is about the advancements in smartphone camera technology."

- **Informative Summarization**: Provides detailed information about the key points in the text. It's more comprehensive than indicative summaries.
- **Example**: "This article discusses the new features of the iPhone 13's camera, including improved low-light performance and the new night mode."

Generic vs Query-Focused Summarization

- **Generic Summarization**: Summarizes the entire document without a specific focus. It tries to cover all major points but is not tailored for a specific question.
- **Example**: A summary of an entire news article or research paper.
- **Query-Focused Summarization**: Summarizes the text with a focus on answering a specific query or addressing a particular aspect of the document.
- **Example**: "What new features does the iPhone 13 camera offer?" – The summary focuses only on answering this question.

Applications of Text Summarization

- **News**: Summarizing long news articles for quicker consumption (e.g., **Google News**).
- **Legal Domain**: Summarizing long legal documents to highlight key points (e.g., **Case law summaries**).
- **Research Papers**: Automatically summarizing academic papers to assist researchers in quickly understanding the content.
- **Customer Reviews**: Summarizing customer feedback for products or services (e.g., **Amazon Reviews**).

Applications of Text Summarization

Which type of summarization (Extractive / Abstractive) would be most effective for each of the following?

- **News Articles:**
- **Legal Documents:**
- **Research Papers:**
- **Customer Reviews:**

Answer:
News Articles: Extractive Summarization
Legal Documents: Extractive Summarization
Research Papers: Abstractive Summarization
Customer Reviews: Abstractive Summarization.

Generic vs Query-Focused Summarization

Slide 6: Key Takeaways and Future Directions
Content:
Text Summarization plays a crucial role in reducing information overload and improving accessibility.
Extractive methods are simpler but often lack coherence, while **abstractive** methods offer more natural and human-like summaries.
Indicative summaries are suitable for general overviews, while **informative** summaries are better for detailed insights.
Query-focused summaries enhance user experience by answering specific questions.
Future Directions:
Moving towards **more sophisticated abstractive models** like **GPT-3** and **BERT** for better quality summaries.
Applications will continue to grow in domains like **business intelligence**, **education**, and **customer service**.

Topic Representation Approaches

Topic Representation Approaches (TRA)

- **Topic Representation refers** to the techniques used to represent and identify the topics within a collection of documents.
- These approaches are fundamental in information retrieval, text summarization, and natural language processing tasks.

Why is it important?

- Helps categorize large text datasets.
- Extracts the main topics from documents to aid in clustering, summarization, and classification tasks.

Term Frequency (TF) and Term Frequency-Inverse Document Frequency (TF-IDF)

Term Frequency (TF): Measures how frequently a term appears in a document. It is computed as:

$$\text{TF}(t, d) = \frac{\text{Number of times term } t \text{ appears in document } d}{\text{Total number of terms in document } d}$$

Term Frequency-Inverse Document Frequency (TF-IDF): Weighs terms by their frequency in a document and how rare the term is across the entire corpus. It reduces the weight of common words (like "is", "the") and increases the weight of rare, significant words.

$$\text{TF–IDF}(t, d) = TF(t, d) \times \log\left(\frac{\text{Total number of documents}}{\text{Number of documents containing } t}\right)$$

Example Problem for TF and TF-IDF

Consider the following 3 documents: **Doc1**: "Machine learning is a field of AI."
Doc2: "Deep learning is a subfield of machine learning."
Doc3: "AI is transforming industries."
1. Calculate the **TF** for the term "machine" in **Doc1**.
2. Calculate the **TF-IDF** for the term "learning" in **Doc2** using a corpus of 3 documents.

TF Calculation:
TF for "machine" in **Doc1** = 1 (since "machine" appears once out of 6 words in Doc1).

TF-IDF Calculation:
TF for "learning" in **Doc2** = 1/6 (appears once in Doc2, total 6 words).
IDF for "learning" = $\log\left(\frac{3}{2}\right)$, since "learning" appears in 2 out of 3 documents.
TF-IDF = TF * IDF = 0.068

Latent Semantic Analysis (LSA)

LSA is a technique for extracting the underlying meaning or topics in a set of documents by analyzing patterns of word co-occurrence.

LSA working:

1. Create a term-document matrix.
2. Apply **Singular Value Decomposition (SVD)** to reduce the dimensions and extract latent topics.
3. The reduced matrix reveals patterns of associations between terms, thus identifying topics in the text.

SVD breaks down the term-document matrix into singular values, enabling dimensionality reduction & capturing hidden semantic structures.

Example: Given a set of news articles, LSA can identify topics like **"Politics"**, **"Technology"**, and **"Economy**" based on word co-occurrences.

Latent Dirichlet Allocation (LDA)

LDA is a generative probabilistic model that assumes each document is a mixture of topics and each topic is a mixture of words.
It is widely used for **topic modeling**.

LDA working:

1. Each document has a mix of topics.
2. Each topic has a probability distribution over words.
3. LDA assigns each word in a document to a topic based on its likelihood.

LDA is typically trained using - Markov Chain Monte Carlo method to infer the topic distributions.

Example: Use LDA on a collection of blog posts to discover latent topics like **"AI"**, **"Healthcare"**, **"Finance"** from the text.

Topic Representation Approaches

Topic Modelling in Summarization

- ❖ Topic modeling techniques like **LDA** and **LSA** can be used to identify the main themes in a document.
- ❖ These topics can be used as the basis for generating summaries, ensuring that the most important content is included.

Summarization systems can use topic modelling to extract key sentences or paragraphs that are most representative of the main topics.

Example: Given a set of research papers, LDA can identify the main topics like **"Deep Learning"** or **"Computer Vision"**, and the summary would include sentences that describe these topics.

Semantic Embeddings (Word2Vec, GloVe, BERT)

Semantic Embeddings (Word2Vec, GloVe, BERT)

- **Word2Vec**: A model that represents words in a continuous vector space, capturing semantic relationships between words. Words with similar meanings have similar vector representations.
- **GloVe**: Similar to Word2Vec, but it uses matrix factorization techniques to learn word vectors by aggregating global word-word co-occurrence statistics.
- **BERT**: A transformer-based model that generates contextualized embeddings. Unlike Word2Vec and GloVe, BERT understands the meaning of a word based on the context of the sentence.

Semantic Embeddings (Word2Vec, GloVe, BERT)

Semantic Embeddings (Word2Vec, GloVe, BERT)

Technical Inference:
Word2Vec and **GloVe** are pre-trained on large corpora like Wikipedia, while **BERT** is fine-tuned for specific tasks and generates dynamic word embeddings.

Example: **BERT embeddings** can be used to improve topic modeling, as it understands the context in which a word appears (e.g., the word "bank" in **finance** vs **river** contexts).

Example Problem for TF and TF-IDF

Identifying Key Terms in a Large Corpus Using TF-IDF

Problem: You have a large corpus of product reviews for a new smartphone. After processing the corpus, you identify the following terms that appear frequently:

Doc1: "This smartphone has an amazing camera and battery life."
Doc2: "The camera is great but the battery drains fast."
Doc3: "Camera quality is top-notch, but the phone heats up quickly."

1. Calculate the **TF** and **TF-IDF** for the term **"camera"** in **Doc1**, **Doc2**, and **Doc3**.
2. Analyze which terms have the highest **TF-IDF** values across the entire corpus and what this indicates about their significance.

Example Problem for TF and TF-IDF

Solution to Identifying Key Terms in a Large Corpus Using TF-IDF

Step 1: Calculate the TF for the Term "camera" in Each Document

TF (Term Frequency) measures how often a term appears in a document relative to the total number of terms in that document. The formula for **TF** is:

$$\text{TF} = \frac{\textit{Number of times term t appears in document d}}{\textit{Total number of terms in document d}}$$

TF Calculation for "camera"

Doc1: "This smartphone has an amazing camera and battery life."

Total number of terms in Doc1 = 7 ; "camera" appears 1 time.

TF(camera, Doc1) = $\frac{1}{7} = 0.1429$

Doc2: "The camera is great but the battery drains fast."

Total number of terms in Doc2 = 8 ; "camera" appears 1 time.

TF(camera, Doc2) = $\frac{1}{8} = 0.125$

Doc3: "Camera quality is top-notch, but the phone heats up quickly."

Total number of terms in Doc3 = 9 ; "camera" appears 1 time.

TF(camera, Doc3) = $\frac{1}{9} = 0.1111$

Example Problem for TF and TF-IDF

Solution to Identifying Key Terms in a Large Corpus Using TF-IDF

Step 2: Calculate the IDF for the Term "camera" ; IDF measures how important a term is within the entire corpus.

If a term appears in many documents, it has a lower IDF. The formula for **IDF** is:

$$\text{IDF}(t) = \log\left(\frac{\textit{Total number of documents}}{\textit{Number of documents containing term } t}\right)$$

Total number of documents = 3 (Doc1, Doc2, Doc3)

"camera" appears in all three documents, so the number of documents containing "camera" = 3.

Since "camera" appears in all the documents, the **IDF** value is 0, which means it's not a discriminating term in the corpus.

Step 3: Calculate the TF-IDF for the Term "camera"

The formula for **TF-IDF** is: $\text{TF}-\text{IDF} = TF \times IDF$

Since the **IDF** for "camera" is 0, the **TF-IDF** for "camera" in all documents will be 0:

TF-IDF(camera, Doc1) = 0.1429 × 0 = 0 ; **TF-IDF(camera, Doc2)** = 0.125 × 0 = 0

TF-IDF(camera, Doc3) = 0.1111 × 0 = 0

Example Problem for TF and TF-IDF

Step 4: Analyze the Significance of Terms Based on TF-IDF :

- Since **"camera"** appears in all documents and has an **IDF of 0**, it is a common term across the corpus and doesn't help in distinguishing documents from each other.

- This is reflected in the **TF-IDF** value of 0.

- Let's consider another term, such as **"battery"**, which might be more specific in identifying the features mentioned in the reviews.

Example Problem for TF and TF-IDF

Step 5: Calculate TF and TF-IDF for "battery"

TF Calculation for "battery"

Doc1: "This smartphone has an amazing camera and battery life."

Total terms = 7 ; "battery" appears 1 time. ; **TF(battery, Doc1)** = $\frac{1}{7} = 0.1429$

Doc2: "The camera is great but the battery drains fast."

Total terms = 8. ; "battery" appears 1 time. ; **TF(battery, Doc2)** = $\frac{1}{8} = 0.125$

Doc3: "Camera quality is top-notch, but the phone heats up quickly."

Total terms = 9. ; "battery" does not appear. ; **TF(battery, Doc3)** = 0

IDF Calculation for "battery" - "battery" appears in Doc1 and Doc2, so it is present in 2 out of 3 documents. IDF(battery) $= \log\left(\frac{3}{2}\right) \approx 0.1761$

TF-IDF Calculation for "battery"

TF-IDF(battery, Doc1) = 0.1429 × 0.1761 = 0.0251.

TF-IDF(battery, Doc2) = 0.125 × 0.1761 = 0.0220

TF-IDF(battery, Doc3) = 0 × 0.1761 = 0

Example Problem for TF and TF-IDF

Solution to Identifying Key Terms in a Large Corpus Using TF-IDF

Step 6: Summary of Results

TF-IDF for "camera": Since "camera" is common in all documents, its **TF-IDF** is 0, indicating it is not significant for distinguishing documents.

TF-IDF for "battery": **"battery"** has higher **TF-IDF** values in **Doc1** and **Doc2** compared to **Doc3**. This indicates that "battery" is more significant in these two documents.

- **TF-IDF** helps identify key terms in a document by considering both their frequency and their rarity across the entire corpus.
- A term that appears in many documents (like "camera") has a low **TF-IDF**, while a term that appears in fewer documents (like "battery") can have a higher **TF-IDF**, making it more important for differentiating documents.

Indicator Representations and Machine Learning for Summarization

Indicator Representations and Machine Learning for Summarization

- **Extractive Summarization:** This technique involves selecting sentences directly from the original document to form a summary. It is based on identifying the most important sentences that represent the essence of the text.
- The goal is to choose the most relevant sentences based on specific features to create a concise summary.

Indicator Representations and Machine Learning for Summarization

Features Used for Extractive Summarization

- **Sentence Position**: Sentences appearing at the beginning or the end of a document are often more important.
- **Cue Phrases**: Certain words or phrases (e.g., "in conclusion", "thus", "therefore") often indicate important sentences that summarize key ideas.
- **Length**: Longer sentences may contain more information, but overly long sentences can also be noise, so balance is needed.
- **Similarity**: Sentences that are more similar to the overall theme of the document are likely to be important.
- **TF-IDF**: Measures how important a term is in the document relative to its occurrence in the entire corpus.
- **Topic Relevance**: Sentences relevant to the main topics or themes in the document are prioritized.

Machine Learning-Based Summarization Approaches

Classification Approach:

- **How it works**: Each sentence is treated as a "class", and the system classifies whether it should be included in the summary.
- **Feature Engineering**: Features like sentence position, length & similarity are extracted and fed into a classifier (e.g., **SVM** or **Logistic Regression**) to predict important sentences.

Graph-Based Methods:

- **TextRank**: A graph-based ranking algorithm that assigns a score to each sentence based on its importance, using sentence similarity as the edge weights.
- **Algorithm**: Sentences are nodes, and edges represent similarity between sentences. **PageRank** algorithm is applied to determine sentence importance.

Neural Methods for Summarization

- **Seq2Seq (Sequence-to-Sequence)**: A deep learning method where an encoder-decoder architecture is used to generate summaries. The model is trained to map the input text to a summary.
- **Attention Mechanism**: Helps the model focus on different parts of the input sequence, improving the generation of summaries by weighting parts of the text more heavily.
- **Transformers (T5, BART)**:
  - **T5**: A text-to-text transformer model that can be fine-tuned for summarization tasks.
  - **BART**: A model that combines the best of both **BERT** (bidirectional encoding) and **GPT** (autoregressive decoding) to improve the quality of generated summaries.

Evaluation of Summarization Models

- **ROUGE Metrics**: A set of metrics that compares the overlap between the generated summary and reference summaries. Key metrics include:
  - **ROUGE-N**: Measures n-gram overlap (e.g., ROUGE-1 measures unigram overlap).
  - **ROUGE-L**: Measures the longest common subsequence between the summary and reference.
- **BLEU Score**: Used for evaluating the quality of machine-generated text (mostly for machine translation but can be adapted for summarization). It measures the precision of n-grams in the generated summary compared to reference summaries.

Summary of Methods for Summarization

Summary of Methods:

- **Extractive Summarization** focuses on selecting important sentences using features like position, cue phrases, and similarity.
- **Machine Learning Methods** (e.g., classification, graph-based methods like TextRank) help automate the extraction of key sentences.
- **Neural Methods** (e.g., Seq2Seq, Attention, T5, and BART) offer advanced techniques for generating high-quality abstractive summaries.

Evaluation: Use **ROUGE** and **BLEU** scores to evaluate the quality of the summaries generated by different models.

Future of Summarization: With advancements in **Transformers**, abstractive summarization is becoming more accurate and context-aware.

Thank you